TucanGo — Guía Definitiva de Estudio del Backend Java (UML v5.0)

Autor: Gian Marco Castañeda Samboni (Rol: Backend / Arquitectura Java)
Curso: Lógica & Algoritmos II — Universidad de la Amazonia
Propósito: Documento de estudio exhaustivo y detallado, línea por línea y concepto por concepto, de todo el backend implementado para TucanGo. Cero detalles obviados.

📑 Tabla de Contenidos

1. Fundamentos de Programación Orientada a Objetos en Java
2. Estructura de Paquetes
3. Enumeraciones (Enums) — Tipos de Datos con Valores Fijos
4. 3.1 EstadoViaje.java
5. 3.2 EstadoPago.java
6. 3.3 MetodoPago.java
7. 3.4 RolAdmin.java
8. Jerarquía de Herencia: La Superclase Base
9. 4.1 Persona.java (Clase Abstracta)
10. Subclases de Persona (Especialización / Polimorfismo)
11. 5.1 Estudiante.java
12. 5.2 Motorista.java
13. 5.3 Administrador.java
14. Entidades de Dominio e Información
15. 6.1 Moto.java
16. 6.2 Viaje.java
17. 6.3 Pago.java
18. 6.4 Calificacion.java
19. Capa de Servicios (Lógica de Negocio Desacoplada)
20. 7.1 ServicioAutenticacion.java
21. 7.2 ServicioReportes.java
22. Puntos de Entrada y Demostración Ejecutable
23. 8.1 Main.java
24. 8.2 DemoTucanGo.java
25. Preguntas Frecuentes y Trampas de Examen

1. Fundamentos de Programación Orientada a Objetos en Java

Antes de entrar clase por clase, estos son los 4 pilares y conceptos clave aplicados en este código:

1. **Encapsulamiento:** Ocultar los detalles internos de una clase mediante modificadores de acceso (`private`, `protected`) y permitir su lectura/modificación controlada a través de métodos públicos (*getters* y *setters*).
2. **Herencia (`extends`):** Mecanismo que permite a una clase hija (*subclase*) heredar atributos y métodos de una clase padre (*superclase*). Modela la relación "*Es-Un*" (Un `Estudiante` *es una* `Persona`).
3. **Polimorfismo:** Capacidad de tratar objetos de diferentes subclases a través del tipo de su superclase común (por ejemplo, registrar tanto a un `Estudiante` como a un `Motorista` en un método que recibe una `Persona`).
4. **Abstracción (`abstract`):** Definir una plantilla conceptual que no puede ser instanciada directamente (no se puede hacer `new Persona()`), obligando a que se creen tipos específicos (`new Estudiante(...)`).
5. **Modificadores de Acceso:**
6. `private`: Solo visible dentro de la misma clase.
7. `protected`: Visible dentro de la misma clase, sus subclases y clases del mismo paquete.
8. `public`: Visible desde cualquier parte del proyecto.
9. **Constructores:** Métodos especiales que se ejecutan automáticamente al crear una instancia (`new`) para inicializar los atributos del objeto.
10. **Palabra clave `this`:** Referencia a la instancia actual del objeto (ej. `this.nombre = nombre;` diferencia el atributo del objeto del parámetro que entra a la función).
11. **Palabra clave `super`:** Hace referencia directa al constructor o métodos de la clase padre (superclase).

---

2. Estructura de Paquetes

El proyecto sigue la convención estándar de nombres de paquete en Java basada en dominios invertidos:

```
co.edu.uniamazonia.logica2
├── modelo/       -> Entidades del dominio, datos y enums.
├── servicio/     -> Lógica de procesamiento y servicios desacoplados.
├── Main.java     -> Entrada completa con menú interactivo y consola.
└── DemoTucanGo.java -> Demostración secuencial limpia del ciclo de vida.
```

---

3. Enumeraciones (Enums) — Tipos de Datos con Valores Fijos

Un `enum` es un tipo de dato especial que restringe una variable para que solo pueda tomar uno de un conjunto de valores constantes predefinidos. Evita errores de tipeo típicos de usar `String` sueltos (como escribir `"ACEPTADO"` vs `"aceptado"`).

3.1 `EstadoViaje.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/EstadoViaje.java`

```
package co.edu.uniamazonia.logica2.modelo;

public enum EstadoViaje {
    SOLICITADO,
    ACEPTADO,
    EN_CURSO,
    FINALIZADO,
    CANCELADO
}
```

🔍 Explicación Detallada:

- `SOLICITADO`: El estudiante pidió el viaje, pero ningún motorista lo ha tomado aún.
- `ACEPTADO`: Un motorista con documentos al día aceptó la solicitud.
- `EN_CURSO`: El motorista recogió al estudiante y están en trayecto.
- `FINALIZADO`: Llegaron al destino y el estudiante marcó la llegada segura.
- `CANCELADO`: El viaje fue abortado por alguna de las partes.

3.2 `EstadoPago.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/EstadoPago.java`

```
package co.edu.uniamazonia.logica2.modelo;

public enum EstadoPago {
    PENDIENTE,
    PAGADO_REPORTADO,
    CONFIRMADO_RECIBIDO
}
```

🔍 Explicación Detallada:

Modela el protocolo de **doble confirmación de confianza**: - `PENDIENTE`: El viaje terminó pero aún no se ha efectuado el pago. - `PAGADO_REPORTADO`: El estudiante indica que ya entregó el efectivo o transfirió por Nequi. - `CONFIRMADO_RECIBIDO`: El motorista valida que tiene el dinero en su mano o en su cuenta.

3.3 `MetodoPago.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/MetodoPago.java`

```
package co.edu.uniamazonia.logica2.modelo;

public enum MetodoPago {
    EFECTIVO,
    NEQUI
}
```

🔍 **Explicación Detallada:**

Define las dos únicas modalidades de pago soportadas en el entorno universitario de Florencia/Amazonia.

3.4 `RolAdmin.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/RolAdmin.java`

```java
package co.edu.uniamazonia.logica2.modelo;

public enum RolAdmin {
    DESARROLLADOR,
    BIENESTAR_UNIVERSITARIO
}
```

🔍 **Explicación Detallada:**

Diferencia los tipos de administradores: - `DESARROLLADOR`**:** Acceso a mantenimiento del sistema y logs técnicos. - `BIENESTAR_UNIVERSITARIO`**:** Acceso a reportes estadísticos y analítica de seguridad de los estudiantes.

---

4. Jerarquía de Herencia: La Superclase Base

4.1 `Persona.java` (Clase Abstracta)

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/Persona.java`

```java
package co.edu.uniamazonia.logica2.modelo;

public abstract class Persona {

    protected String identificacion;
    protected String nombre;
    protected String telefono;
    protected String correoInstitucional;

    public Persona(String identificacion, String nombre, String telefono, String correoInstitucional) {
        this.identificacion = identificacion;
        this.nombre = nombre;
        this.telefono = telefono;
        this.correoInstitucional = correoInstitucional;
    }

    public String obtenerIdentificacion() {
        return this.identificacion;
    }

    public String obtenerNombreCompleto() {
        return this.nombre;
    }

    public String getIdentificacion() {
        return identificacion;
    }

    public void setIdentificacion(String identificacion) {
        this.identificacion = identificacion;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getTelefono() {
        return telefono;
    }

    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    public String getCorreoInstitucional() {
        return correoInstitucional;
    }

    public void setCorreoInstitucional(String correoInstitucional) {
        this.correoInstitucional = correoInstitucional;
    }

    @Override
    public String toString() {
        return "Persona{" +
                "identificacion='" + identificacion + ''' +
```

```
                ", nombre='" + nombre + ''' +
                ", telefono='" + telefono + ''' +
                ", correoInstitucional='" + correoInstitucional + ''' +
                '}';
    }
}
```

🔍 Desglose Técnico:

1. `public abstract class Persona`:
2. `abstract`: Significa que es una clase incompleta conceptualmente por sí misma. No se puede hacer `new Persona()`. Su único fin es ser heredada.
3. **Atributos** `protected`:
4. `protected String identificacion;`, `nombre`, `telefono`, `correoInstitucional`: Al ser `protected`, las clases hijas (`Estudiante`, `Motorista`, `Administrador`) pueden acceder a ellos directamente sin necesidad de llamar a un *getter*, pero quedan protegidos contra el exterior.
5. **Constructor** `public Persona(...)`:
6. Inicializa los 4 atributos comunes requeridos para cualquier miembro del campus.
7. **Métodos de Negocio (`obtenerIdentificacion`, `obtenerNombreCompleto`)**:
8. Métodos semánticos exigidos por el diseño UML para mayor legibilidad.
9. **Getters y Setters**:
10. Permiten consultar y actualizar de forma controlada cada campo.
11. `@Override public String toString()`:
12. Sobrescribe el método estándar de Java `Object.toString()` para retornar una representación legible en texto del objeto al imprimirlo en consola.

---

5. Subclases de Persona (Especialización / Polimorfismo)

5.1 `Estudiante.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/Estudiante.java`

```java
package co.edu.uniamazonia.logica2.modelo;

import java.util.ArrayList;
import java.util.List;

public class Estudiante extends Persona {

    private String codigoEstudiantil;
    private List<Viaje> viajesSolicitados;

    public Estudiante(String identificacion, String nombre, String telefono, String correoInstitucional,
        super(identificacion, nombre, telefono, correoInstitucional);
        this.codigoEstudiantil = codigoEstudiantil;
        this.viajesSolicitados = new ArrayList<>();
    }

    public boolean solicitarViaje(String origen, String destino) {
        if (origen == null || origen.isBlank() || destino == null || destino.isBlank()) {
            return false;
        }
        String codigoV = "VIA-" + (this.viajesSolicitados.size() + 1);
        Viaje nuevoViaje = new Viaje(codigoV, origen, destino, 0.0);
        this.viajesSolicitados.add(nuevoViaje);
        return true;
    }

    public boolean solicitarViaje(String origen, String destino, double tarifa) {
        if (origen == null || origen.isBlank() || destino == null || destino.isBlank() || tarifa < 0) {
            return false;
        }
        String codigoV = "VIA-" + (this.viajesSolicitados.size() + 1);
        Viaje nuevoViaje = new Viaje(codigoV, origen, destino, tarifa);
        this.viajesSolicitados.add(nuevoViaje);
        return true;
    }

    public boolean marcarLlegadaSegura(String codigoViaje) {
        if (codigoViaje == null || codigoViaje.isBlank()) {
            return false;
        }
        for (Viaje v : viajesSolicitados) {
            if (v.getCodigoViaje().equalsIgnoreCase(codigoViaje)) {
                v.finalizarViaje();
                return true;
            }
        }
        return false;
    }

    public String getCodigoEstudiantil() {
        return codigoEstudiantil;
    }

    public void setCodigoEstudiantil(String codigoEstudiantil) {
        this.codigoEstudiantil = codigoEstudiantil;
    }

    public List<Viaje> getViajesSolicitados() {
        return viajesSolicitados;
    }
```

```
    @Override
    public String toString() {
        return "Estudiante{" +
                "codigoEstudiantil='" + codigoEstudiantil + ''' +
                ", identificacion='" + identificacion + ''' +
                ", nombre='" + nombre + ''' +
                ", telefono='" + telefono + ''' +
                ", correoInstitucional='" + correoInstitucional + ''' +
                ", totalViajes=" + viajesSolicitados.size() +
                '}';
    }
}
```

🔍 Desglose Técnico:

1. `public class Estudiante extends Persona`:
2. `extends`: Aplica herencia. `Estudiante` hereda automáticamente nombre, cédula, teléfono y correo.
3. **Atributos Específicos**:
4. `codigoEstudiantil`: Matrícula universitaria única (ej. `20231001`).
5. `viajesSolicitados`: Colección dinámica `List<Viaje>` que almacena el historial de viajes que este estudiante ha pedido.
6. `super(...)` **en el constructor**:
7. Llama al constructor de `Persona` para inicializar los datos heredados. Luego inicializa su `codigoEstudiantil` e instancia el `ArrayList`.
8. **Sobrecarga de Métodos (`solicitarViaje`)**:
9. Existen dos versiones del método `solicitarViaje`:
    - Versión 1: Solo recibe `origen` y `destino` (tarifa por defecto en 0.0).
    - Versión 2: Recibe `origen`, `destino` y `tarifa`.
10. Valida que los textos no sean nulos ni vacíos (`.isBlank()`) y que la tarifa no sea negativa (`tarifa < 0`).
11. Genera automáticamente un código correlativo: `"VIA-" + (size + 1)`.
12. `marcarLlegadaSegura(String codigoViaje)`:
13. Recorre el historial con un bucle `for-each`, busca el viaje que coincida con el código (ignorando mayúsculas/minúsculas con `.equalsIgnoreCase`) y ejecuta `v.finalizarViaje()`.

5.2 `Motorista.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/Motorista.java`

```java
package co.edu.uniamazonia.logica2.modelo;

import java.util.ArrayList;
import java.util.List;

public class Motorista extends Persona {

    private boolean disponible;
    private List<Moto> motos;
    private List<Viaje> viajesAtendidos;

    public Motorista(String identificacion, String nombre, String telefono, String correoInstitucional) {
        super(identificacion, nombre, telefono, correoInstitucional);
        this.disponible = true;
        this.motos = new ArrayList<>();
        this.viajesAtendidos = new ArrayList<>();
    }

    public Motorista(String identificacion, String nombre, String telefono, String correoInstitucional, b
        super(identificacion, nombre, telefono, correoInstitucional);
        this.disponible = disponible;
        this.motos = new ArrayList<>();
        this.viajesAtendidos = new ArrayList<>();
    }

    public void registrarMoto(Moto moto) {
        if (moto != null) {
            if (this.motos.isEmpty()) {
                moto.setActiva(true);
            }
            this.motos.add(moto);
        }
    }

    public boolean seleccionarMotoActiva(String placa) {
        if (placa == null || placa.isBlank()) {
            return false;
        }
        boolean encontrada = false;
        for (Moto m : this.motos) {
            if (m.getPlaca().equalsIgnoreCase(placa)) {
                encontrada = true;
                break;
            }
        }
        if (!encontrada) {
            return false;
        }
        for (Moto m : this.motos) {
            m.setActiva(m.getPlaca().equalsIgnoreCase(placa));
        }
        return true;
    }

    public Moto obtenerMotoActiva() {
        for (Moto m : this.motos) {
            if (m.isActiva()) {
                return m;
            }
        }
```

```
        return null;
    }

    public boolean aceptarViaje(String codigoViaje) {
        if (this.disponible && verificarDocumentos() && codigoViaje != null && !codigoViaje.isBlank()) {
            this.disponible = false;
            return true;
        }
        return false;
    }

    public boolean aceptarViaje(Viaje viaje) {
        if (viaje != null && this.disponible && verificarDocumentos()) {
            viaje.setEstado(EstadoViaje.ACEPTADO);
            this.viajesAtendidos.add(viaje);
            this.disponible = false;
            return true;
        }
        return false;
    }

    public void registrarDisponibilidad(boolean estado) {
        this.disponible = estado;
    }

    public boolean verificarDocumentos() {
        Moto motoActiva = obtenerMotoActiva();
        return motoActiva != null && motoActiva.validarSoat();
    }

    public boolean isDisponible() {
        return disponible;
    }

    public void setDisponible(boolean disponible) {
        this.disponible = disponible;
    }

    public List<Moto> getMotos() {
        return motos;
    }

    public List<Viaje> getViajesAtendidos() {
        return viajesAtendidos;
    }

    @Override
    public String toString() {
        Moto motoActiva = obtenerMotoActiva();
        String infoMoto = (motoActiva != null) ? motoActiva.getPlaca() + " (" + motoActiva.getModelo() +
        return "Motorista{" +
                "identificacion='" + identificacion + ''' +
                ", nombre='" + nombre + ''' +
                ", telefono='" + telefono + ''' +
                ", disponible=" + disponible +
                ", motoActiva=" + infoMoto +
                ", totalMotos=" + motos.size() +
                ", totalViajes=" + viajesAtendidos.size() +
                '}';
    }
}
```

🔍 **Desglose Técnico:**

1. **Soporte Multi-Moto (`List<Moto> motos`)**:
2. En lugar de limitar al motorista a una sola moto, maneja una lista (`1..*`).
3. **Lógica de Moto Activa**:
4. `registrarMoto(Moto moto)`: Si la lista está vacía, la primera moto que registra se marca automáticamente como activa (`moto.setActiva(true)`).
5. `seleccionarMotoActiva(String placa)`: Permite cambiar de vehículo. Busca la placa, y si existe, activa esa moto y desactiva las demás en un solo pase.
6. **Regla de Oro de Seguridad (`verificarDocumentos`)**:
7. Un motorista **NO puede aceptar ningún viaje** si no tiene una moto activa o si el SOAT de su moto activa está vencido (`motoActiva.validarSoat()`).
8. `aceptarViaje(...)`:
9. Valida: `this.disponible == true` Y `verificarDocumentos() == true`.
10. Cambia el estado del viaje a `EstadoViaje.ACEPTADO`, guarda el viaje en su lista `viajesAtendidos` y se pone en estado no disponible (`this.disponible = false`).

5.3 `Administrador.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/Administrador.java`

```java
package co.edu.uniamazonia.logica2.modelo;

public class Administrador extends Persona {

    private RolAdmin rol;

    public Administrador(String identificacion, String nombre, String telefono, String correoInstituciona
        super(identificacion, nombre, telefono, correoInstitucional);
        this.rol = (rol != null) ? rol : RolAdmin.BIENESTAR_UNIVERSITARIO;
    }

    public void consultarReportesDemanda() {
        System.out.println("[ADMIN] Consultando reportes de demanda y destinos frecuentes para rol: " + t
    }

    public void gestionarUsuarios() {
        System.out.println("[ADMIN] Gestionando usuarios y permisos en la plataforma como: " + this.rol);
    }

    public RolAdmin getRol() {
        return rol;
    }

    public void setRol(RolAdmin rol) {
        this.rol = rol;
    }

    @Override
    public String toString() {
        return "Administrador{" +
                "rol=" + rol +
                ", identificacion='" + identificacion + ''' +
                ", nombre='" + nombre + ''' +
                ", correoInstitucional='" + correoInstitucional + ''' +
                '}';
    }
}
```

🔍 **Desglose Técnico:**

- Especializa a `Persona` para personal administrativo.
- Incluye el atributo tipado `- rol : RolAdmin`.
- En el constructor usa un operador ternario `(rol != null) ? rol : ...` para evitar referencias nulas por defecto.

---

6. Entidades de Dominio e Información

6.1 `Moto.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/Moto.java`

```java
package co.edu.uniamazonia.logica2.modelo;

public class Moto {

    private String placa;
    private String marca;
    private String modelo;
    private int cilindraje;
    private boolean soatVigente;
    private String numeroSoat;
    private boolean activa;

    public Moto(String placa, String marca, String modelo, int cilindraje, boolean soatVigente, String nu
        this.placa = placa;
        this.marca = marca;
        this.modelo = modelo;
        this.cilindraje = cilindraje;
        this.soatVigente = soatVigente;
        this.numeroSoat = numeroSoat;
        this.activa = false;
    }

    public Moto(String placa, String marca, String modelo, int cilindraje, boolean soatVigente, String nu
        this.placa = placa;
        this.marca = marca;
        this.modelo = modelo;
        this.cilindraje = cilindraje;
        this.soatVigente = soatVigente;
        this.numeroSoat = numeroSoat;
        this.activa = activa;
    }

    public boolean validarSoat() {
        return this.soatVigente && this.numeroSoat != null && !this.numeroSoat.isBlank();
    }

    public String getPlaca() {
        return placa;
    }

    public void setPlaca(String placa) {
        this.placa = placa;
    }

    public String getMarca() {
        return marca;
    }

    public void setMarca(String marca) {
        this.marca = marca;
    }

    public String getModelo() {
        return modelo;
    }

    public void setModelo(String modelo) {
        this.modelo = modelo;
    }
```

```
    public int getCilindraje() {
        return cilindraje;
    }

    public void setCilindraje(int cilindraje) {
        this.cilindraje = cilindraje;
    }

    public boolean isSoatVigente() {
        return soatVigente;
    }

    public void setSoatVigente(boolean soatVigente) {
        this.soatVigente = soatVigente;
    }

    public String getNumeroSoat() {
        return numeroSoat;
    }

    public void setNumeroSoat(String numeroSoat) {
        this.numeroSoat = numeroSoat;
    }

    public boolean isActiva() {
        return activa;
    }

    public void setActiva(boolean activa) {
        this.activa = activa;
    }

    @Override
    public String toString() {
        return "Moto{" +
                "placa='" + placa + ''' +
                ", marca='" + marca + ''' +
                ", modelo='" + modelo + ''' +
                ", cilindraje=" + cilindraje +
                ", soatVigente=" + soatVigente +
                ", numeroSoat='" + numeroSoat + ''' +
                ", activa=" + activa +
                '}';
    }
}
```

🔍 Desglose Técnico:

- Desacopla el vehículo del conductor (cumpliendo con el Principio de Responsabilidad Única - SRP).
- `validarSoat()`: Verifica que el booleano `soatVigente` sea verdadero Y que exista un número de póliza registrado y no vacío.
- `isSoatVigente()` / `isActiva()`: Convención JavaBeans en métodos getter de tipo `boolean` (empiezan por `is` en vez de `get`).

6.2 `Viaje.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/Viaje.java`

```java
package co.edu.uniamazonia.logica2.modelo;

import java.util.ArrayList;
import java.util.List;

public class Viaje {

    private String codigoViaje;
    private String origen;
    private String destino;
    private double tarifa;
    private EstadoViaje estado;
    private String fechaHora;
    private Pago pago;
    private List<Calificacion> calificaciones;

    public Viaje(String codigoViaje, String origen, String destino, double tarifa) {
        this.codigoViaje = codigoViaje;
        this.origen = origen;
        this.destino = destino;
        this.tarifa = tarifa;
        this.estado = EstadoViaje.SOLICITADO;
        this.fechaHora = java.time.LocalDateTime.now().toString();
        this.calificaciones = new ArrayList<>();
    }

    public double calcularTarifa() {
        if (this.tarifa <= 0) {
            this.tarifa = 4000.0; // Tarifa base universitaria por defecto
        }
        return this.tarifa;
    }

    public void iniciarViaje() {
        if (this.estado == EstadoViaje.ACEPTADO) {
            this.estado = EstadoViaje.EN_CURSO;
        }
    }

    public void finalizarViaje() {
        this.estado = EstadoViaje.FINALIZADO;
    }

    public void cancelarViaje() {
        this.estado = EstadoViaje.CANCELADO;
    }

    public boolean agregarCalificacion(Calificacion calificacion) {
        if (calificacion != null && this.calificaciones.size() < 2) {
            this.calificaciones.add(calificacion);
            return true;
        }
        return false;
    }

    public String getCodigoViaje() {
        return codigoViaje;
    }

    public void setCodigoViaje(String codigoViaje) {
```

```
        this.codigoViaje = codigoViaje;
    }

    public String getOrigen() {
        return origen;
    }

    public void setOrigen(String origen) {
        this.origen = origen;
    }

    public String getDestino() {
        return destino;
    }

    public void setDestino(String destino) {
        this.destino = destino;
    }

    public double getTarifa() {
        return tarifa;
    }

    public void setTarifa(double tarifa) {
        this.tarifa = tarifa;
    }

    public EstadoViaje getEstado() {
        return estado;
    }

    public void setEstado(EstadoViaje estado) {
        this.estado = estado;
    }

    public String getFechaHora() {
        return fechaHora;
    }

    public void setFechaHora(String fechaHora) {
        this.fechaHora = fechaHora;
    }

    public Pago getPago() {
        return pago;
    }

    public void setPago(Pago pago) {
        this.pago = pago;
    }

    public List<Calificacion> getCalificaciones() {
        return calificaciones;
    }

    @Override
    public String toString() {
        return "Viaje{" +
                "codigoViaje='" + codigoViaje + ''' +
                ", origen='" + origen + ''' +
                ", destino='" + destino + ''' +
```

```
                ", tarifa=" + tarifa +
                ", estado=" + estado +
                ", fechaHora='" + fechaHora + ''' +
                ", pago=" + (pago != null ? pago.getEstado() : "Sin pago") +
                ", totalCalificaciones=" + calificaciones.size() +
                '}';
    }
}
```

🔍 Desglose Técnico:

- **Entidad Núcleo Transaccional:** Coordina toda la vida de un servicio.
- **Máquina de Estados:** Controla transiciones (`SOLICITADO` -> `ACEPTADO` -> `EN_CURSO` -> `FINALIZADO` o `CANCELADO`).
- **Control de Multiplicidad `0..2` en Calificaciones (`agregarCalificacion`):**
- En un viaje solo existen máximo **2 calificaciones** (la del Estudiante hacia el Motorista, y la del Motorista hacia el Estudiante). El método valida `this.calificaciones.size() < 2` antes de añadir.

6.3 `Pago.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/Pago.java`

```java
package co.edu.uniamazonia.logica2.modelo;

public class Pago {

    private double valor;
    private MetodoPago metodo;
    private EstadoPago estado;
    private boolean pagadoPorEstudiante;
    private boolean confirmadoPorMotorista;
    private String fechaHora;

    public Pago(double valor, MetodoPago metodo) {
        this.valor = valor;
        this.metodo = (metodo != null) ? metodo : MetodoPago.EFECTIVO;
        this.estado = EstadoPago.PENDIENTE;
        this.pagadoPorEstudiante = false;
        this.confirmadoPorMotorista = false;
        this.fechaHora = java.time.LocalDateTime.now().toString();
    }

    public void reportarPagoEstudiante(MetodoPago metodo) {
        if (metodo != null) {
            this.metodo = metodo;
        }
        this.pagadoPorEstudiante = true;
        this.estado = EstadoPago.PAGADO_REPORTADO;
    }

    public boolean confirmarRecepcionMotorista() {
        if (this.pagadoPorEstudiante) {
            this.confirmadoPorMotorista = true;
            this.estado = EstadoPago.CONFIRMADO_RECIBIDO;
            return true;
        }
        return false;
    }

    public double getValor() {
        return valor;
    }

    public void setValor(double valor) {
        this.valor = valor;
    }

    public MetodoPago getMetodo() {
        return metodo;
    }

    public void setMetodo(MetodoPago metodo) {
        this.metodo = metodo;
    }

    public EstadoPago getEstado() {
        return estado;
    }

    public void setEstado(EstadoPago estado) {
        this.estado = estado;
    }
```

```
    public boolean isPagadoPorEstudiante() {
        return pagadoPorEstudiante;
    }

    public void setPagadoPorEstudiante(boolean pagadoPorEstudiante) {
        this.pagadoPorEstudiante = pagadoPorEstudiante;
    }

    public boolean isConfirmadoPorMotorista() {
        return confirmadoPorMotorista;
    }

    public void setConfirmadoPorMotorista(boolean confirmadoPorMotorista) {
        this.confirmadoPorMotorista = confirmadoPorMotorista;
    }

    public String getFechaHora() {
        return fechaHora;
    }

    public void setFechaHora(String fechaHora) {
        this.fechaHora = fechaHora;
    }

    @Override
    public String toString() {
        return "Pago{" +
                "valor=" + valor +
                ", metodo=" + metodo +
                ", estado=" + estado +
                ", pagadoPorEstudiante=" + pagadoPorEstudiante +
                ", confirmadoPorMotorista=" + confirmadoPorMotorista +
                '}';
    }
}
```

🔍 Desglose Técnico:

- Modela un pago peer-to-peer (P2P) de mutuo acuerdo.
- `confirmarRecepcionMotorista()`: Solo puede pasar a `CONFIRMADO_RECIBIDO` si previamente `pagadoPorEstudiante` es verdadero.

6.4 `Calificacion.java`

Ubicación: `src/co/edu/uniamazonia/logica2/modelo/Calificacion.java`

```java
package co.edu.uniamazonia.logica2.modelo;

public class Calificacion {

    private double puntaje;
    private String comentario;
    private String rolEmisor; // "ESTUDIANTE" o "MOTORISTA"
    private String fechaHora;

    public Calificacion(double puntaje, String comentario, String rolEmisor) {
        this.puntaje = normalizarPuntaje(puntaje);
        this.comentario = comentario;
        this.rolEmisor = (rolEmisor != null && !rolEmisor.isBlank()) ? rolEmisor.toUpperCase() : "ANONIMO
        this.fechaHora = java.time.LocalDateTime.now().toString();
    }

    private double normalizarPuntaje(double p) {
        if (p < 1.0) {
            return 1.0;
        }
        if (p > 5.0) {
            return 5.0;
        }
        return Math.round(p * 10.0) / 10.0; // Redondea a 1 decimal
    }

    public void registrarCalificacion(double puntaje, String comentario, String emisor) {
        this.puntaje = normalizarPuntaje(puntaje);
        this.comentario = comentario;
        if (emisor != null && !emisor.isBlank()) {
            this.rolEmisor = emisor.toUpperCase();
        }
    }

    public double obtenerPuntaje() {
        return this.puntaje;
    }

    public double getPuntaje() {
        return puntaje;
    }

    public void setPuntaje(double puntaje) {
        this.puntaje = normalizarPuntaje(puntaje);
    }

    public String getComentario() {
        return comentario;
    }

    public void setComentario(String comentario) {
        this.comentario = comentario;
    }

    public String getRolEmisor() {
        return rolEmisor;
    }

    public void setRolEmisor(String rolEmisor) {
        this.rolEmisor = rolEmisor;
```

```
    }

    public String getFechaHora() {
        return fechaHora;
    }

    public void setFechaHora(String fechaHora) {
        this.fechaHora = fechaHora;
    }

    @Override
    public String toString() {
        return "Calificacion{" +
                "puntaje=" + puntaje +
                ", comentario='" + comentario + ''' +
                ", rolEmisor='" + rolEmisor + ''' +
                ", fechaHora='" + fechaHora + ''' +
                '}';
    }
}
```

🔍 **Desglose Técnico:**

- `normalizarPuntaje(double p)` **(Método Privado)**:
- Garantiza que el puntaje siempre esté acotado en el rango legal de **1.0 a 5.0**.
- Si un usuario ingresa `0.5`, se ajusta a `1.0`. Si ingresa `7.0`, se ajusta a `5.0`.
- Usa la fórmula matemática `Math.round(p * 10.0) / 10.0` para truncar a 1 solo decimal (ej. `4.85` -> `4.9`).

---

7. Capa de Servicios (Lógica de Negocio Desacoplada)

7.1 `ServicioAutenticacion.java`

Ubicación: `src/co/edu/uniamazonia/logica2/servicio/ServicioAutenticacion.java`

```java
package co.edu.uniamazonia.logica2.servicio;

import co.edu.uniamazonia.logica2.modelo.Persona;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

public class ServicioAutenticacion {

    private Map<String, Persona> usuariosRegistrados;
    private Map<String, String> credenciales;
    private Set<String> sesionesActivas;

    public ServicioAutenticacion() {
        this.usuariosRegistrados = new HashMap<>();
        this.credenciales = new HashMap<>();
        this.sesionesActivas = new HashSet<>();
    }

    public boolean registrarse(Persona persona, String clave) {
        if (persona == null || clave == null || clave.length() < 4) {
            return false;
        }
        String correo = persona.getCorreoInstitucional();
        if (correo == null || correo.isBlank() || usuariosRegistrados.containsKey(correo)) {
            return false;
        }
        usuariosRegistrados.put(correo, persona);
        credenciales.put(correo, clave);
        return true;
    }

    public boolean iniciarSesion(String correo, String clave) {
        if (correo == null || clave == null) {
            return false;
        }
        if (credenciales.containsKey(correo) && credenciales.get(correo).equals(clave)) {
            sesionesActivas.add(correo);
            return true;
        }
        return false;
    }

    public void cerrarSesion(String identificacionOCorreo) {
        if (identificacionOCorreo != null) {
            sesionesActivas.remove(identificacionOCorreo);
        }
    }

    public boolean estaAutenticado(String correo) {
        return correo != null && sesionesActivas.contains(correo);
    }
}
```

🔍 Desglose Técnico:

- **Estructuras de Datos (`Collections Framework`)**:

- `Map<String, Persona> usuariosRegistrados`: Diccionario clave-valor donde la clave es el correo institucional y el valor es el objeto `Persona` (aprovechando polimorfismo para guardar estudiantes, motoristas o admins).
- `Map<String, String> credenciales`: Mapea correo a contraseña. Valida contraseña mínima de 4 caracteres.
- `Set<String> sesionesActivas`: Conjunto de correos con sesión abierta (búsqueda $O(1)$ sin duplicados).

7.2 `ServicioReportes.java`

Ubicación: `src/co/edu/uniamazonia/logica2/servicio/ServicioReportes.java`

```java
package co.edu.uniamazonia.logica2.servicio;

import co.edu.uniamazonia.logica2.modelo.Viaje;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class ServicioReportes {

    private List<Viaje> historicoViajes;

    public ServicioReportes() {
        this.historicoViajes = new ArrayList<>();
    }

    public void registrarViaje(Viaje viaje) {
        if (viaje != null) {
            this.historicoViajes.add(viaje);
        }
    }

    public List<String> obtenerDestinosMasFrecuentes(int limite) {
        Map<String, Integer> conteo = new HashMap<>();
        for (Viaje v : historicoViajes) {
            String destino = v.getDestino();
            if (destino != null && !destino.isBlank()) {
                conteo.put(destino, conteo.getOrDefault(destino, 0) + 1);
            }
        }
        List<String> reporte = new ArrayList<>();
        conteo.entrySet().stream()
                .sorted((a, b) -> b.getValue().compareTo(a.getValue()))
                .limit(limite > 0 ? limite : 5)
                .forEach(e -> reporte.add(e.getKey() + " (" + e.getValue() + " viajes)"));
        return reporte;
    }

    public List<Viaje> consultarViajesPorDestino(String destino) {
        List<Viaje> filtrados = new ArrayList<>();
        if (destino == null || destino.isBlank()) {
            return filtrados;
        }
        for (Viaje v : historicoViajes) {
            if (v.getDestino() != null && v.getDestino().equalsIgnoreCase(destino)) {
                filtrados.add(v);
            }
        }
        return filtrados;
    }

    public double calcularPorcentajePorSector(String sector) {
        if (historicoViajes.isEmpty() || sector == null || sector.isBlank()) {
            return 0.0;
        }
        long totalSector = historicoViajes.stream()
                .filter(v -> v.getDestino() != null && v.getDestino().toLowerCase().contains(sector.toLow
                .count();
        double porcentaje = ((double) totalSector / historicoViajes.size()) * 100.0;
        return Math.round(porcentaje * 10.0) / 10.0;
```

```
    }
}
```

🔍 **Desglose Técnico:**

- **Inteligencia de Negocio y Datos**:
- `obtenerDestinosMasFrecuentes(int limite)`: Utiliza `Java Streams` y `HashMap` con `.getOrDefault(destino, 0) + 1` para contar frecuencias y ordenar de mayor a menor concurrencia.
- `calcularPorcentajePorSector(String sector)`: Calcula la tasa porcentual de demanda hacia un barrio o campus: $$ ext{Porcentaje} = \left(rac{ ext{Viajes al Sector}}{ ext{Total de Viajes}} ight) imes 100$$

---

8. Puntos de Entrada y Demostración Ejecutable

8.1 `Main.java`

Ubicación: `src/co/edu/uniamazonia/logica2/Main.java` - Es la clase que implementa la interfaz de menú por consola con `java.util.Scanner` para navegación interactiva (solicitar viajes, listar motoristas, ver reportes de Bienestar Universitario).

8.2 `DemoTucanGo.java`

Ubicación: `src/co/edu/uniamazonia/logica2/DemoTucanGo.java` - Es el script de prueba automatizado y determinista que ejecuta de punta a punta: 1. Instanciación de `Estudiante` y `Motorista` (con moto y SOAT). 2. Creación y aceptación de un `Viaje`. 3. Finalización de viaje y reporte de `Pago` con Nequi. 4. Generación de `Calificacion` mutua (5.0 y 4.9).

---

9. Preguntas Frecuentes y Trampas de Examen

❓ P1: ¿Por qué `Persona` es `abstract` y no una clase normal?

> **Respuesta de 5.0:** Porque en el mundo real universitario nadie es solo una "Persona genérica"; en el campus eres un `Estudiante`, un `Motorista` o un `Administrador`. Hacerla abstracta evita que se creen instancias incompletas (`new Persona()`) y obliga al uso de polimorfismo.

❓ P2: ¿Por qué los atributos en `Persona` son `protected` y no `private`?

> **Respuesta:** Para que las clases hijas (`Estudiante`, `Motorista`) puedan acceder a ellos directamente al construir sus propios métodos y representaciones (`toString()`) sin romper el encapsulamiento hacia el resto del paquete.

❓ P3: ¿Cómo maneja el sistema que un motorista tenga más de una moto?

> **Respuesta:** Mediante una lista `List<Moto>` y una bandera booleana `activa`. El método `seleccionarMotoActiva(placa)` garantiza que solo un vehículo esté activo a la vez para los viajes.

❓ P4: ¿Qué pasa si el SOAT de la moto está vencido?

> **Respuesta:** El método `verificarDocumentos()` del `Motorista` evalúa `motoActiva.validarSoat()`. Si retorna `false`, el método `aceptarViaje(...)` rechaza la solicitud de forma inmediata.